

Lecture 18: Sequence Alignment in Linear Space

Date: [date]
Instructor: Aniket Basu Roy
Scribe: Harsha

Overview. The previous lecture set up the SEQUENCE ALIGNMENT problem and gave an $O(mn)$ -time, $O(mn)$ -space dynamic program for it. Today we ask whether that much *space* is really necessary. We first recast the dynamic program as a shortest-path problem in a directed grid graph — a reformulation that costs us nothing but buys a great deal of intuition. We then observe that the recurrence looks back only one row, so two arrays of length $m + 1$ suffice to compute the edit distance in $O(m)$ space. The catch is that this economy destroys exactly the information we wanted: we learn *how far apart* X and Y are, but not *which* characters align with which. The lecture closes by setting up the divide-and-conquer idea (Hirschberg’s algorithm) that recovers the alignment itself without giving up linear space.

1 Recap: The Sequence Alignment Problem

Input. An alphabet Σ ; a gap penalty $\delta > 0$; and a mismatch cost function

$$\alpha : \Sigma \times \Sigma \longrightarrow \mathbb{R}.$$

In particular we may take $\delta = 1$ together with

$$\alpha(p, p) = 0, \quad \alpha(p, q) = 1 \text{ for } p \neq q, \quad p, q \in \Sigma,$$

which recovers the ordinary edit distance. Two strings $X = x_1x_2 \cdots x_m$ and $Y = y_1y_2 \cdots y_n$ over Σ are given.

Definition 1 (Alignment). An *alignment* M of X and Y is a matching between the positions of X and the positions of Y that is *crossing-free*: if $(x_i, y_j) \in M$ and $(x_{i'}, y_{j'}) \in M$ with $i < i'$, then $j < j'$.

So if positions 1, 2, 3 of X and 1, 2, 3 of Y are available and x_1 is matched to y_2 , then x_2 may *not* be matched to y_1 : the two edges would cross. Every position of X and of Y is either matched to exactly one position of the other string, or left unmatched.

Definition 2 (Cost of an alignment).

$$\text{cost}(M) = \delta \cdot |\{a : a \text{ is unmatched in } X \cup Y \text{ w.r.t. } M\}| + \sum_{(x_i, y_j) \in M} \alpha(x_i, y_j).$$

The first term charges δ once for every gap, on either side; the second charges the mismatch cost of every matched pair.

Definition 3 (Edit distance).

$$\text{dist}(X, Y) = \min_{M \text{ an alignment}} \text{cost}(M).$$

Worked example (from last lecture). Take $X = abc$ and $Y = bac$. One alignment matches $x_1 = a$ to $y_2 = a$ and $x_3 = c$ to $y_3 = c$, leaving $x_2 = b$ and $y_1 = b$ unmatched; its cost is 2δ (two gaps) plus $\alpha(a, a) + \alpha(c, c) = 0$. Note that matching $x_2 = b$ to $y_1 = b$ *as well* is forbidden — that edge would cross the edge (x_1, y_2) .

2 The Dynamic Program

Write $f(i, j) = \text{dist}(x_1 \cdots x_i, y_1 \cdots y_j)$, so that $f(m, n) = \text{OPT}(m, n) = \text{dist}(X, Y)$ is the quantity we want.

Base cases. Aligning a prefix against the empty string forces every character to be a gap:

$$f(i, 0) = i\delta, \quad f(0, j) = j\delta.$$

Recurrence. Consider the last characters x_i and y_j . Exactly one of three things happens in an optimal alignment: they are matched to each other, or x_i is unmatched, or y_j is unmatched. (By the crossing-free property, if x_i is

matched at all it can only be matched to y_j .) Hence

$$f(i, j) = \min \begin{cases} \alpha(x_i, y_j) + f(i-1, j-1), & x_i \text{ matched to } y_j, \\ \delta + f(i-1, j), & x_i \text{ unmatched}, \\ \delta + f(i, j-1), & y_j \text{ unmatched}. \end{cases}$$

The second and third cases are symmetric in the two strings.

Filling the $(m+1) \times (n+1)$ table bottom-up, each cell costs $O(1)$, so the algorithm runs in $O(mn)$ time and uses $O(mn)$ space.

Today's question. Given this recurrence, $O(mn)$ time seems hard to beat. But do we really need $O(mn)$ space?

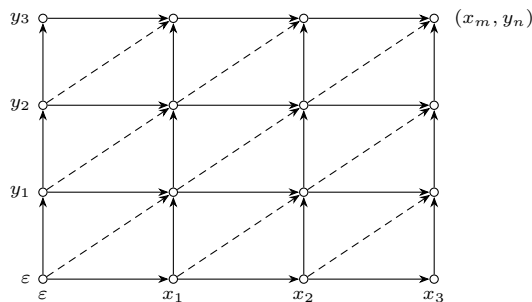
3 Reformulation: Alignment as a Shortest Path

It will pay to view the table as a graph.

Definition 4 (The alignment graph $G_{X,Y}$). The vertex set is the Cartesian product $\{\varepsilon, x_1, \dots, x_m\} \times \{\varepsilon, y_1, \dots, y_n\}$, which we index by pairs (i, j) with $0 \leq i \leq m$, $0 \leq j \leq n$; here ε denotes the empty string, giving the dummy row and column. The directed edges out of each vertex mimic the three cases of the recurrence:

- $(i-1, j) \rightarrow (i, j)$ of weight δ ;
- $(i, j-1) \rightarrow (i, j)$ of weight δ ;
- $(i-1, j-1) \rightarrow (i, j)$ of weight $\alpha(x_i, y_j)$.

On the board, the two axis-parallel families were drawn in yellow (weight δ each, since each such step leaves one character unmatched) and the diagonal family in pink (weight $\alpha(x_i, y_j)$, since such a step matches x_i with y_j). A useful sanity check raised in class: the diagonal edge *entering* vertex (i, j) is the one labelled $\alpha(x_i, y_j)$ — the label is determined by the head of the edge, not its tail.



The graph $G_{X,Y}$ (drawn for $m = n = 3$). Solid edges have weight δ ; dashed diagonal edges entering (i, j) have weight $\alpha(x_i, y_j)$. Every path from $(0, 0)$ to (m, n) is monotone — it never moves left or down.

Lemma 1. For all $0 \leq i \leq m$, $0 \leq j \leq n$, the value $f(i, j)$ equals the length of a shortest path from $(0, 0)$ to (i, j) in $G_{X,Y}$. In particular, $\text{dist}(X, Y)$ is the shortest-path length from $(0, 0)$ to (m, n) .

Proof idea (induction on $i+j$). The base cases $i = 0$ or $j = 0$ are the unique monotone paths along the boundary, of length $j\delta$ and $i\delta$ respectively, agreeing with $f(0, j)$ and $f(i, 0)$. For the inductive step, note that (i, j) has in-degree exactly three: any path reaching (i, j) arrives via $(i-1, j)$, $(i, j-1)$ or $(i-1, j-1)$, paying δ , δ or $\alpha(x_i, y_j)$ on that final edge. Minimising over the three predecessors, each of which has smaller coordinate sum and so satisfies the claim by induction, gives exactly the recurrence for $f(i, j)$. ■

So the three cases of the dynamic program and the three in-edges of a grid vertex are the same statement read twice. Reaching the top-right corner (x_m, y_n) from the bottom-left corner $(\varepsilon, \varepsilon)$ along a cheapest monotone path *is* the alignment problem, and the path itself traces out the alignment: each diagonal edge on it is a matched pair, each axis-parallel edge a gap.

4 Linear Space via Two Arrays

Look again at the recurrence: computing column j of the table requires only column $j - 1$. Nothing older is ever consulted. So we need not store the whole table.

A first attempt, and why it fails. It was suggested in class that since each cell depends on only three values, three variables — and hence $O(1)$ space — should suffice. This is too aggressive. To advance from (i, j) to the next cell we would need $f(i - 1, j)$, $f(i, j - 1)$ and $f(i - 1, j - 1)$ again for *that* cell, and those are different three values; the shortest path may route through cells we have already discarded. Three variables let us walk one cell forward, not sweep the table.

What does work. Keep a constant number of *columns*, not a constant number of *cells*. Maintain two arrays

$$B[0..m][0] \quad \text{and} \quad B[0..m][1],$$

where $B[\cdot][1]$ holds the column currently being computed and $B[\cdot][0]$ the previous one. Initialise $B[i][0] = i\delta$ for $0 \leq i \leq m$ — the boundary values $0, \delta, 2\delta, 3\delta, \dots$ — and then sweep.

Algorithm 1: Space-Efficient-Alignment

Input: Strings $X = x_1 \cdots x_m$, $Y = y_1 \cdots y_n$; gap penalty δ ; mismatch cost α
Output: $\text{dist}(X, Y)$

```

1 for  $i \leftarrow 0$  to  $m$  do  $B[i][0] \leftarrow i\delta$ 
2 for  $j \leftarrow 1$  to  $n$  do
3    $B[0][1] \leftarrow j\delta$ 
4   for  $i \leftarrow 1$  to  $m$  do
5      $B[i][1] \leftarrow \min\{\alpha(x_i, y_j) + B[i-1][0], \delta + B[i-1][1], \delta + B[i][0]\}$ 
6   for  $i \leftarrow 0$  to  $m$  do  $B[i][0] \leftarrow B[i][1]$            // copy: the new column becomes the old
7 return  $B[m][0]$ 
```

Theorem 1. Algorithm 1 computes $\text{dist}(X, Y)$ in $O(mn)$ time and $O(m)$ space.

The time bound is unchanged — every cell is still evaluated once — but the space drops from $O(mn)$ to two arrays of length $m + 1$. Note that we should orient the strings so that $m = \min\{|X|, |Y|\}$, since the space is linear in whichever string indexes the arrays.

5 The Catch: We Lose the Alignment

Algorithm 1 returns a *number*. If you hand me two strings I can now tell you how far apart they are, but I have no idea which character of X is matched with which character of Y . The alignment M itself — the set of matched edges, say $\{(x_1, y_2), \dots\}$ — is gone.

The reason is structural, not incidental. Recovering M means walking the shortest path backwards from (m, n) to $(0, 0)$, and at any moment Algorithm 1 can see only two adjacent columns. It cannot look back. We have thrown away precisely the space in which the path was recorded.

Remark (A proposal from the floor). One suggestion was to keep a second $(m + 1) \times (n + 1)$ table of 0/1 flags, writing 1 on the predecessor that achieved the minimum at each cell, and then backtracking through the flags at the end. This is exactly the standard backpointer construction, and it is correct — but it stores one bit per cell, so it is back to $\Theta(mn)$ space. It solves the wrong problem: we already know how to recover M in $O(mn)$ space. The question is whether we can do it in less.

The state of play.

- $O(mn)$ time, $O(mn)$ space: distance *and* alignment. (Last lecture.)
- $O(mn)$ time, $O(m)$ space: distance only. (Algorithm 1.)
- $O(mn)$ time, $O(m)$ space: distance *and* alignment. (Today's goal.)

6 Towards Divide and Conquer

The way out is to compute the alignment from both ends at once.

Definition 5 (Backward distance). Let $g(i, j)$ denote the length of a shortest path from (i, j) to (m, n) in $G_{X,Y}$.

So g is the backward mirror of f : where $f(i, j)$ is the shortest path from the origin to (i, j) , $g(i, j)$ is the shortest path from (i, j) to the destination. Crucially, g satisfies an entirely symmetric recurrence and can therefore be computed in $O(m)$ space by running Algorithm 1 on the reversed strings.

Lemma 2 (Decomposition). *If a vertex (i, j) lies on some shortest $(0, 0)$ – (m, n) path, then the length of that path is $f(i, j) + g(i, j)$. Consequently, for any fixed j ,*

$$\text{dist}(X, Y) = \min_{0 \leq i \leq m} [f(i, j) + g(i, j)],$$

and any i^ attaining the minimum gives a vertex (i^*, j) lying on some shortest path.*

The instructor flagged that the “if and only if” direction here — that a shortest path can always be cut cleanly at such a vertex — still requires proof, and deferred it.

The algorithm in outline. Take the middle column $j = n/2$ (assume n is a power of two, so the halving is clean). Compute $f(i, n/2)$ for all i by a forward sweep, and $g(i, n/2)$ for all i by a backward sweep — both in $O(m)$ space. Find the index i^* minimising $f(i, n/2) + g(i, n/2)$. By Lemma 2, the vertex $(i^*, n/2)$ lies on a shortest path, so the optimal alignment splits into an alignment of $x_1 \cdots x_{i^*}$ with $y_1 \cdots y_{n/2}$ and an alignment of $x_{i^*+1} \cdots x_m$ with $y_{n/2+1} \cdots y_n$. Recurse on both halves.

Algorithm 2: Divide-and-Conquer-Alignment (sketch)

Input: A rectangle of $G_{X,Y}$ with corners $(0, 0)$ and (m, n)
Output: A shortest $(0, 0)$ – (m, n) path, i.e. an optimal alignment M
1 **if** n is a constant **then** solve directly by the $O(mn)$ -space algorithm and **return**
2 $j \leftarrow n/2$
3 Compute $f(i, j)$ for all i by a forward sweep (Algorithm 1)
4 Compute $g(i, j)$ for all i by a backward sweep on the reversed strings
5 $i^* \leftarrow \arg \min_{0 \leq i \leq m} [f(i, j) + g(i, j)]$
6 Record (i^*, j) as a vertex of the output path
7 Recurse on the rectangle $(0, 0)$ – (i^*, j)
8 Recurse on the rectangle (i^*, j) – (m, n)

Each level of the recursion halves the number of columns: an $m \times n$ problem becomes two problems of $n/2$ columns each, then four of $n/4$, and so on, stopping when the number of columns is constant. The claim to be established next time is that this still runs in $O(mn)$ time — the halving of the column count compensates for the repeated sweeps — while never using more than $O(m)$ space.

7 Left for Next Lecture

Three things were explicitly deferred:

1. A proof of Lemma 2 — that a shortest path really does decompose at a minimiser of $f(i, j) + g(i, j)$, so that the recursion is sound.
2. The space analysis of Algorithm 2: why the recursion does not accumulate more than $O(m)$ space across its levels.
3. The time analysis: solving the recurrence to confirm $O(mn)$ time overall.

Reference. Kleinberg, J., & Tardos, E. (2006). *Algorithm Design*, Sections 6.6–6.7. Pearson/Addison-Wesley.